DS ASSIGNMENT 4-KARISHUMA VIKRAM

1. **Implement a Stack Using Queue Data Structure**

```
 #include <stdio.h>
#define MAX 100

int q1[MAX], q2[MAX];
int front1 = 0, rear1 = -1;
int front2 = 0, rear2 = -1;

void push(int x)
{
   q2[++rear2] = x;

   while (front1 <= rear1)
      q2[++rear2] = q1[front1++];

   // Copy q2 back to q1
   front1 = 0;
   rear1 = -1;

   while (front2 <= rear2)
      q1[++rear1] = q2[front2++];

   front2 = 0;
   rear2 = -1;
}

void pop()
{
   if (front1 > rear1)
   {
```

```
        printf("Stack Underflow\n");
        return;
    }

    printf("Popped element: %d\n", q1[front1++]);
}

void display()
{
    if (front1 > rear1)
    {
        printf("Stack is empty\n");
        return;
    }

    printf("Stack elements: ");
    for (int i = front1; i <= rear1; i++)
        printf("%d ", q1[i]);

    printf("\n");
}

int main()
{
    push(10);
    push(20);
    push(30);

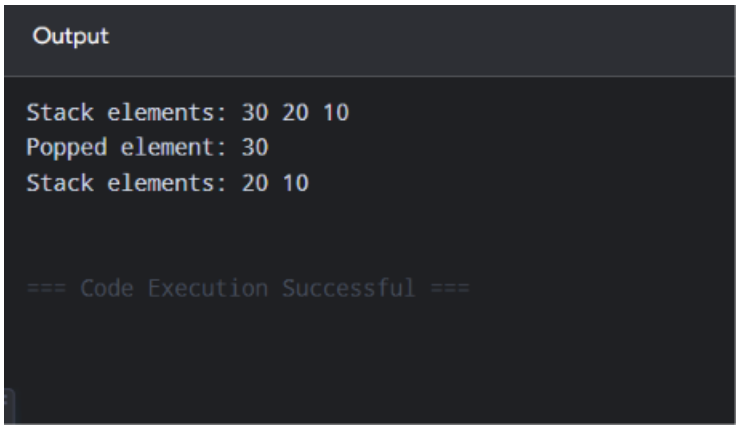
    display();

    pop();
```

```
    display();


    return 0;
}
```

Output



2. **Implement a Queue Using Stack Data Structure**

```
#include <stdio.h>
#define MAX 100

int s1[MAX], s2[MAX];
int top1 = -1, top2 = -1;

void enqueue(int x)
{
   s1[++top1] = x;
}

void dequeue()
{
   if (top1 == -1 && top2 == -1)
   {
      printf("Queue Underflow\n");
      return;
   }

   if (top2 == -1)
   {
      while (top1 != -1)
         s2[++top2] = s1[top1--];
   }

   printf("Deleted element: %d\n", s2[top2--]);
}
```

```
void display()
{
   if (top1 == -1 && top2 == -1)
   {
      printf("Queue is empty\n");
      return;
   }

   printf("Queue elements: ");

   for (int i = top2; i >= 0; i--)
      printf("%d ", s2[i]);

   for (int i = 0; i <= top1; i++)
      printf("%d ", s1[i]);

   printf("\n");
}

int main()
{
   enqueue(10);
   enqueue(20);
   enqueue(30);

   display();

   dequeue();
   display();

   return 0;
}
```

Output

```
Output

Queue elements: 10 20 30
Deleted element: 10
Queue elements: 20 30


=== Code Execution Successful ===
```

3. **Write a C Program Using Stack to Manage the History of Web Pages Visited**

```
#include <stdio.h>
#include <string.h>
```

```
#define MAX 100

char history[MAX][100];
int top = -1;

void visitPage(char page[])
{
   if (top == MAX - 1)
   {
      printf("History is full!\n");
      return;
   }

   strcpy(history[++top], page);
   printf("Visited: %s\n", page);
}

void back()
{
   if (top == -1)
   {
      printf("No previous page available.\n");
      return;
   }

   printf("Going back from: %s\n", history[top--]);

   if (top != -1)
      printf("Current page: %s\n", history[top]);
   else
      printf("No page in history.\n");
}

void displayHistory()
{
   if (top == -1)
   {
      printf("History is empty.\n");
      return;
   }

   printf("\nWeb Page History:\n");

   for (int i = top; i >= 0; i--)
      printf("%s\n", history[i]);
}

int main()
```

```
{
   visitPage("google.com");
   visitPage("youtube.com");
   visitPage("wikipedia.org");
   visitPage("github.com");

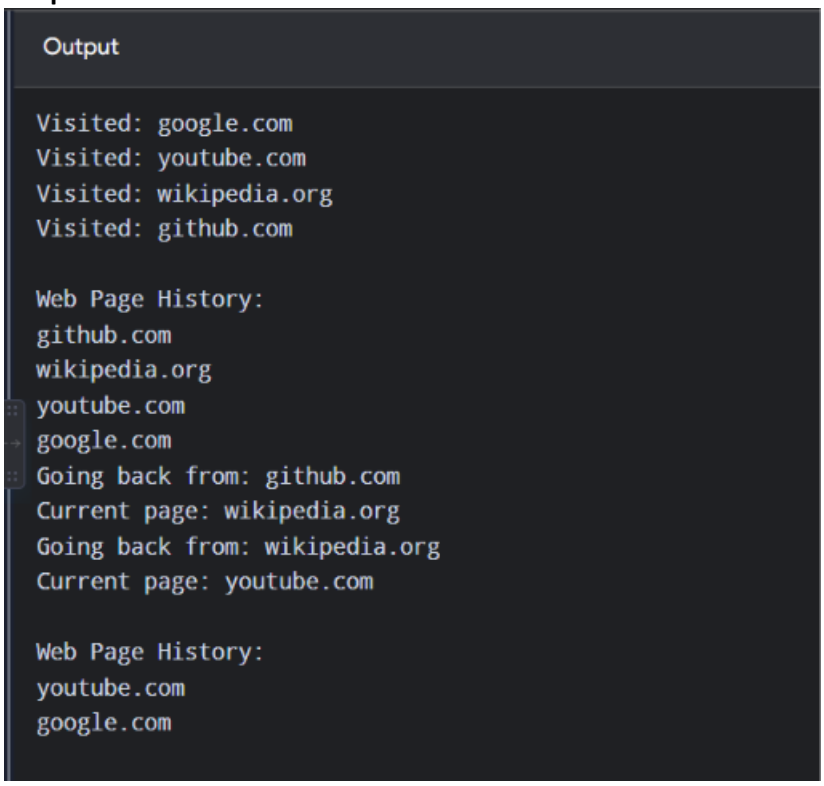
   displayHistory();

   back();
   back();

   displayHistory();

   return 0;
}
```

Output



4. **Write a C program to sort a stack using a temporary stack.**

```
#include <stdio.h>

#define MAX 100

int stack[MAX], temp[MAX];
int top = -1, ttop = -1;

void push(int s[], int *top, int value)
{
   s[++(*top)] = value;
```

```
}

int pop(int s[], int *top)
{
   return s[(*top)--];
}

void sortStack()
{
   while (top != -1)
   {
      int x = pop(stack, &top);

      while (ttop != -1 && temp[ttop] > x)
      {
         push(stack, &top, pop(temp, &ttop));
      }

      push(temp, &ttop, x);
   }

   while (ttop != -1)
   {
      push(stack, &top, pop(temp, &ttop));
   }
}

void display()
{
   printf("Sorted Stack: ");

   for (int i = top; i >= 0; i--)
      printf("%d ", stack[i]);

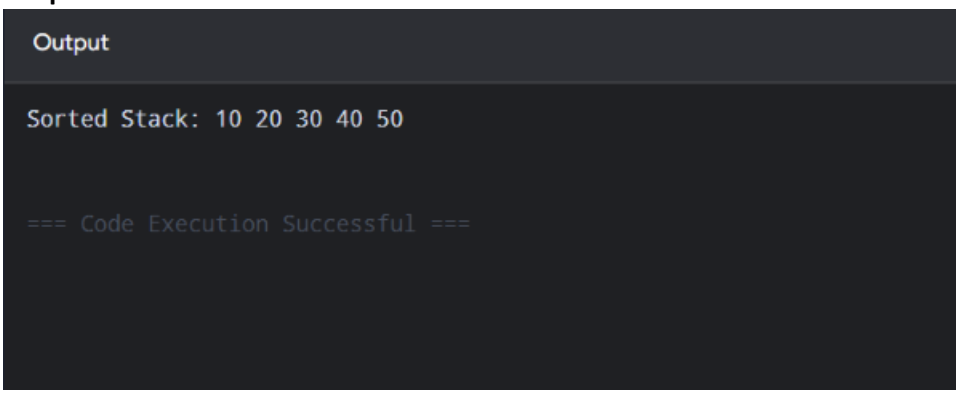
   printf("\n");
}

int main()
{
   push(stack, &top, 30);
   push(stack, &top, 10);
   push(stack, &top, 50);
   push(stack, &top, 20);
   push(stack, &top, 40);

   sortStack();
   display();
```

```
    return 0;
}
```

Output



5. Stock Span Problem

```
#include <stdio.h>

#define MAX 100

void calculateSpan(int price[], int n)
{
    int span[MAX];
    int stack[MAX];
    int top = -1;

    span[0] = 1;
    stack[++top] = 0;

    for (int i = 1; i < n; i++)
    {
        while (top != -1 && price[stack[top]] <= price[i])
            top--;

        if (top == -1)
            span[i] = i + 1;
        else
```

```
            span[i] = i - stack[top];

        stack[++top] = i;
    }

    printf("Stock Span: ");

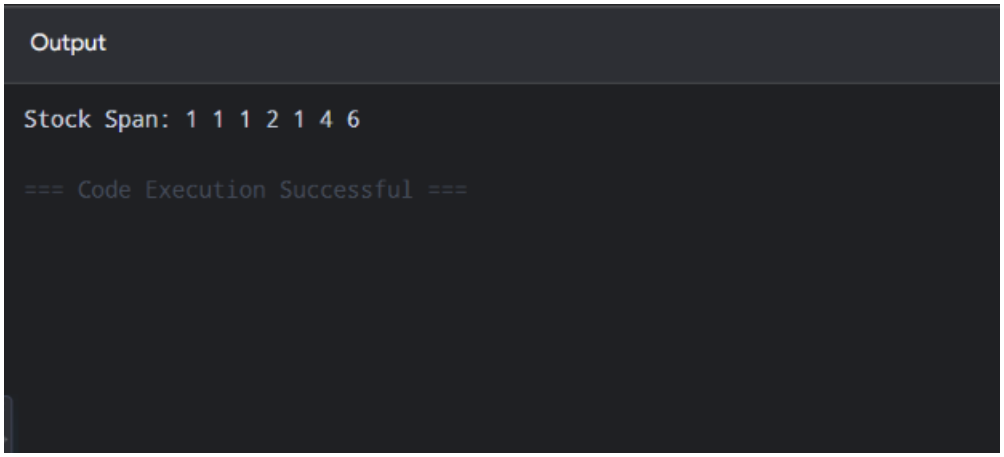
    for (int i = 0; i < n; i++)
        printf("%d ", span[i]);
}

int main()
{
    int price[] = {100, 80, 60, 70, 60, 75, 85};
    int n = 7;

    calculateSpan(price, n);

    return 0;
}
```

Output



6. **Given an array, find the nearest smaller element to the left of each element. If no smaller element exists, return -1. Implement it using a stack.**

```
#include <stdio.h>

#define MAX 100

void nearestSmaller(int arr[], int n)
{
   int stack[MAX];
   int top = -1;

   printf("Output: ");

   for (int i = 0; i < n; i++)
   {
      while (top != -1 && stack[top] >= arr[i])
         top--;

      if (top == -1)
         printf("-1 ");
      else
         printf("%d ", stack[top]);

      stack[++top] = arr[i];
   }
}

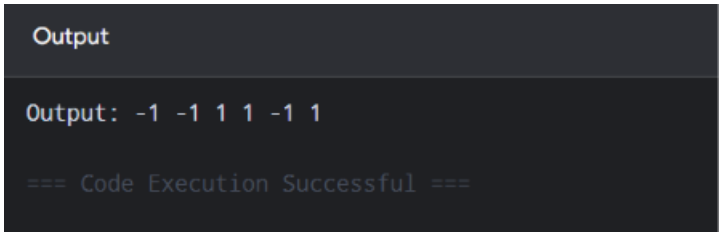
int main()
{
   int arr[] = {2, 1, 3, 2, 1, 7};
```

```
    int n = 6;

    nearestSmaller(arr, n);

    return 0;
}
```

Output



7. Pattern containing I and D

```
#include <stdio.h>

#define MAX 100

void printPattern(char pattern[])
{
    int stack[MAX];
    int top = -1;
    int num = 1;

    for (int i = 0; pattern[i] != '\0'; i++)
    {
        stack[++top] = num++;

        if (pattern[i] == 'I')
        {
            while (top != -1)
                printf("%d", stack[top--]);
        }
    }

    // Push the last number
    stack[++top] = num;

    while (top != -1)
        printf("%d", stack[top--]);

    printf("\n");
}
```

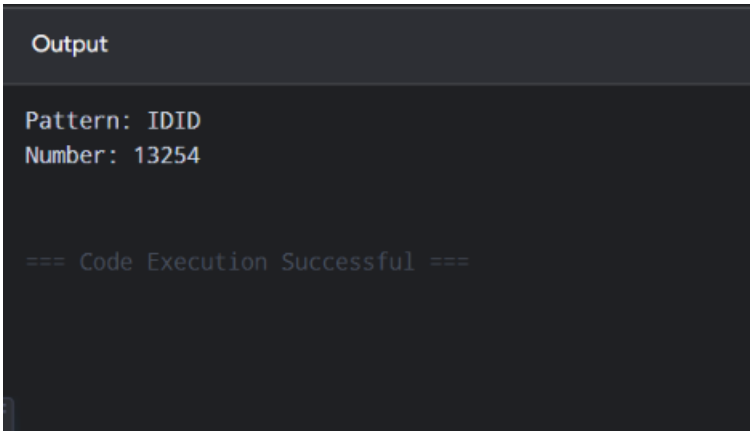
```
int main()
{
   char pattern[] = "IDID";

   printf("Pattern: %s\n", pattern);
   printf("Number: ");

   printPattern(pattern);

   return 0;
}
```

Output



8. **Minimum Sum of Two Numbers**

```
#include <stdio.h>

void sort(int a[], int n)
{
   int i, j, temp;

   for (i = 0; i < n - 1; i++)
   {
      for (j = 0; j < n - i - 1; j++)
      {
         if (a[j] > a[j + 1])
         {
            temp = a[j];
            a[j] = a[j + 1];
            a[j + 1] = temp;
         }
      }
   }
}

int main()
```

```
{
   int a[] = {6, 8, 4, 5, 2, 3};
   int n = 6;
   int num1 = 0, num2 = 0;
   int i;

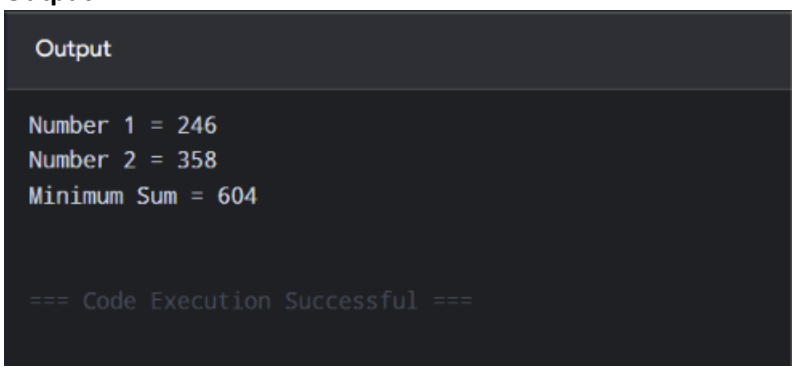
   sort(a, n);

   for (i = 0; i < n; i++)
   {
      if (i % 2 == 0)
         num1 = num1 * 10 + a[i];
      else
         num2 = num2 * 10 + a[i];
   }

   printf("Number 1 = %d\n", num1);
   printf("Number 2 = %d\n", num2);
   printf("Minimum Sum = %d\n", num1 + num2);

   return 0;
}
```

Output



9. **First Negative Integer in Every Window**

```
#include <stdio.h>

int main()
{
   int a[] = {-8, 2, 3, -6, 1};
   int n = 5;
   int k = 2;

   int queue[100];
   int front = 0, rear = 0;
   int i;

   for (i = 0; i < k; i++)
```

```
    {
        if (a[i] < 0)
        {
            queue[rear++] = i;
        }
    }

    for (i = k; i <= n; i++)
    {

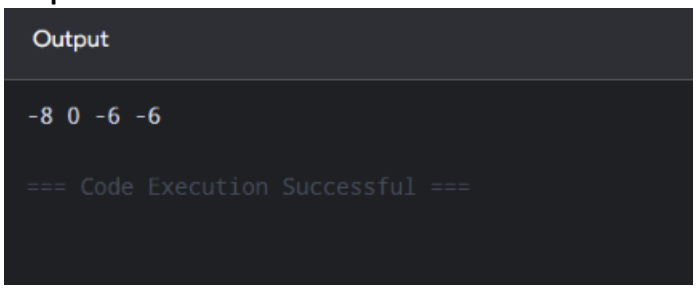
        while (front < rear && queue[front] < i - k)
        {
            front++;
        }

        if (front == rear)
            printf("0 ");
        else
            printf("%d ", a[queue[front]]);


        if (i < n && a[i] < 0)
        {
            queue[rear++] = i;
        }
    }

    return 0;
}
```

Output



10. Given a matrix where:

0 → Empty cell

1 → Fresh orange

2 → Rotten orange

Every rotten orange makes its adjacent fresh oranges rotten in 1 unit of time.

Find the minimum time required to rot all oranges.

If some fresh orange can never become rotten, return -1.

```c
#include <stdio.h>

#define MAX 100

typedef struct
{
   int row;
   int col;
} Node;

int main()
{
   int grid[MAX][MAX];
   Node queue[MAX * MAX];

   int m, n;
   int front = 0, rear = 0;
   int fresh = 0;
   int time = 0;

   int i, j;

   printf("Enter rows and columns: ");
   scanf("%d %d", &m, &n);

   printf("Enter matrix:\n");

   for (i = 0; i < m; i++)
   {
      for (j = 0; j < n; j++)
      {
```

```
            scanf("%d", &grid[i][j]);

            if (grid[i][j] == 2)
            {
                queue[rear].row = i;
                queue[rear].col = j;
                rear++;
            }
            else if (grid[i][j] == 1)
            {
                fresh++;
            }
        }
    }

    /* Directions:
       up, down, left, right */
    int dr[] = {-1, 1, 0, 0};
    int dc[] = {0, 0, -1, 1};

    while (front < rear && fresh > 0)
    {
        int size = rear - front;

        /* One level of BFS = one second */
        for (i = 0; i < size; i++)
        {
            int r = queue[front].row;
            int c = queue[front].col;
            front++;
```

```
            int d;

            for (d = 0; d < 4; d++)
            {
                int nr = r + dr[d];
                int nc = c + dc[d];

                if (nr >= 0 && nr < m &&
                    nc >= 0 && nc < n &&
                    grid[nr][nc] == 1)
                {
                    grid[nr][nc] = 2;
                    fresh--;

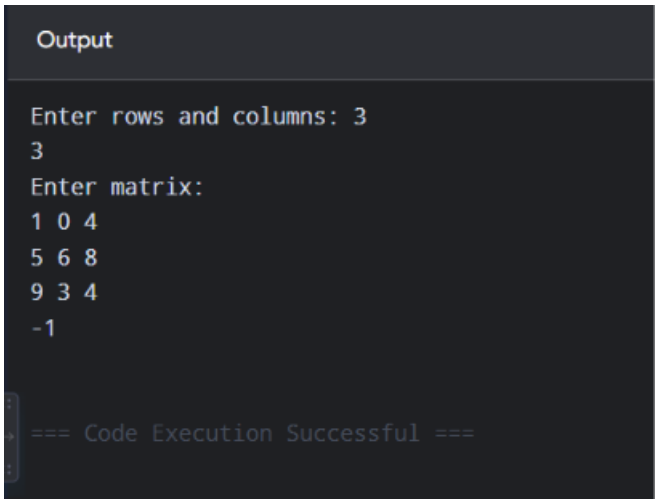
                    queue[rear].row = nr;
                    queue[rear].col = nc;
                    rear++;
                }
            }
        }

        time++;
    }

    if (fresh > 0)
        printf("-1\n");
    else
        printf("%d\n", time);

    return 0;
}
```

Output


Output
Enter rows and columns: 3
3
Enter matrix:
1 0 4
5 6 8
9 3 4
-1
=== Code Execution Successful ===